

INTERVIEW ANSWERS — PART 5

JavaScript & React · System Design · Behavioral & HR · AI & ML

SECTION 12: JAVASCRIPT & REACT (27 Questions)

Q1. Difference in reassign and redeclaration

Reassign: change the VALUE stored in an existing variable. All mutable variables allow this.

Redeclare: declare a NEW variable with the SAME name in the same scope.

var: allows BOTH reassign and redeclare.

let: allows reassign, but NOT redeclare.

const: allows NEITHER reassign nor redeclare.

▶ Reassign vs Redeclare examples:

```
// var - allows both (problematic - can silently overwrite)
var x = 1;
var x = 2;    // redeclare - no error with var (dangerous)
x = 3;        // reassign - allowed

// let - reassign yes, redeclare no
let y = 1;
// let y = 2;  // SyntaxError: Identifier 'y' has already been declared
y = 2;        // reassign - OK

// const - neither
const z = 1;
// const z = 2; // SyntaxError: Identifier 'z' already declared
// z = 2;       // TypeError: Assignment to constant variable

// IMPORTANT: const on objects - reference is constant, PROPERTIES can change
const user = { name: 'Vamshi' };
// user = {};    // TypeError - cannot reassign the reference
user.name = 'Reddy'; // OK - modifying a property, not reassigning the variable
```

💡 *const does not make the object immutable — it only prevents reassigning the VARIABLE. Use `Object.freeze(user)` if you need deep immutability.*

Q2. var vs let vs const

var: function-scoped (not block-scoped), hoisted to top of function as undefined, allows redeclaration. AVOID in modern JS.

let: block-scoped ({} boundary), temporal dead zone (TDZ — not accessible before declaration), no redeclaration. Use for mutable variables.

const: block-scoped, must initialise at declaration, no reassign or redeclaration. DEFAULT choice for all variables.

▶ Scoping differences:

```
// var - function scoped (leaks out of if/for blocks)
function varExample() {
  if (true) {
    var a = 10;    // declared inside if-block
  }
  console.log(a);  // 10 - var leaks out of the if-block!
}

// let - block scoped
function letExample() {
  if (true) {
    let b = 10;
  }
  // console.log(b); // ReferenceError: b is not defined (stays in block)
```

```

}

// var hoisting – declaration moved to top, initialisation stays
console.log(hoisted); // undefined (not ReferenceError – var was hoisted)
var hoisted = 5;
console.log(hoisted); // 5

// let temporal dead zone – exists but not accessible
// console.log(inTDZ); // ReferenceError: Cannot access before initialization
let inTDZ = 5; // TDZ ends here

// Best practice: use const always, let when you must reassign, never var

```

Q3. Types of object creation in JS

▶ 5 ways to create objects:

```

// 1. Object literal (most common)
const user1 = { name: 'Vamshi', role: 'Developer' };

// 2. Constructor function (pre-class ES5 style)
function Person(name, age) {
  this.name = name;
  this.age = age;
}
const user2 = new Person('Vamshi', 30);

// 3. ES6 Class (syntactic sugar over constructor functions)
class Employee {
  constructor(name, dept) {
    this.name = name;
    this.dept = dept;
  }
  greet() { return `Hi, I'm ${this.name}`; }
}
const user3 = new Employee('Vamshi', 'Engineering');

// 4. Object.create() – sets prototype explicitly
const proto = { greet() { return `Hi I am ${this.name}`; } };
const user4 = Object.create(proto); // user4's prototype IS proto
user4.name = 'Vamshi';
console.log(user4.greet()); // Hi I am Vamshi

// 5. Factory function – returns plain object, no 'new'
function createUser(name, role) {
  return { name, role, greet() { return `Hi, I'm ${name}`; } };
}
const user5 = createUser('Vamshi', 'Dev');

```

🔗 *Factory functions avoid "new" keyword mistakes and do not expose prototype. Classes are standard for OOP. Object.create() is useful for prototype-based inheritance without classes.*

Q4. What is hoisting in JS?

Hoisting: JavaScript's behaviour of moving DECLARATIONS (not initialisations) to the TOP of their scope before code executes.

var declarations → hoisted as undefined.

Function declarations → hoisted COMPLETELY (both name and body).

let and const → hoisted but in TDZ (temporal dead zone) — not accessible until declaration line.

▶ Hoisting behaviour:

```

// var hoisting – declaration moved up, but value is undefined until assignment
console.log(name); // undefined (NOT ReferenceError – var was hoisted)
var name = 'Vamshi';
console.log(name); // 'Vamshi'

// Function declaration hoisting – ENTIRE function is hoisted
greet(); // 'Hello!' – works BEFORE the function definition

```

```
function greet() { console.log('Hello!'); }

// Function EXPRESSION – only variable name is hoisted (as undefined)
// sayHello(); // TypeError: sayHello is not a function
var sayHello = function() { console.log('Hi!'); };
// var sayHello is hoisted as undefined, calling undefined() throws TypeError

// let/const – hoisted but in TDZ
// console.log(age); // ReferenceError: Cannot access 'age' before initialization
let age = 25; // TDZ ends here – safe to access after this line
```

Q5. What is temporal dead zone (TDZ)?

The TDZ is the period between entering a scope (block/function) and the let/const declaration being evaluated. During TDZ: the variable EXISTS in scope (hoisted) but accessing it throws ReferenceError.

This prevents the confusing "undefined" behaviour you get with var.

▶ TDZ illustration:

```
{
  // TDZ for 'price' STARTS here (block entered)
  console.log(typeof price); // ReferenceError! (NOT undefined like var)

  // ... code here ...

  let price = 100; // TDZ ENDS here – declaration reached
  console.log(price); // 100 – now safe
}

// Why TDZ is GOOD – catches bugs early
function calculateTotal(qty) {
  // console.log(unitPrice); // Error immediately – tells you the bug
  let unitPrice = 50;
  return qty * unitPrice;
}

// With var – same bug silently returns NaN (undefined * 3)
function badVersion(qty) {
  console.log(unitPrice); // undefined – silent bug!
  var unitPrice = 50;
  return qty * unitPrice; // returns NaN – confusing
}
```

Q6. What is closure in JS?

A closure is a function that REMEMBERS the variables from its outer scope even after the outer function has finished executing.

This works because the inner function keeps a reference to the outer function's variable environment (lexical scope).

▶ Closure examples:

```
// Basic closure – inner function accesses outer variable after outer returns
function makeCounter() {
  let count = 0; // outer variable – will be "closed over"
  return function() { // inner function – closure
    count++; // still accesses 'count' from outer scope
    return count;
  };
}

const counter = makeCounter(); // makeCounter returns inner function, count=0
console.log(counter()); // 1 – count is still in memory!
console.log(counter()); // 2
console.log(counter()); // 3

const counter2 = makeCounter(); // NEW closure – independent count
console.log(counter2()); // 1 – starts fresh
```

```
// Practical use: data privacy (simulating private variables)
function BankAccount(initialBalance) {
  let balance = initialBalance; // private – not accessible directly

  return {
    deposit: (amount) => { balance += amount; },
    withdraw: (amount) => { if (amount <= balance) balance -= amount; },
    getBalance: () => balance // only way to read balance
  };
}

const account = BankAccount(1000);
account.deposit(500);
console.log(account.getBalance()); // 1500
// console.log(account.balance); // undefined – private via closure
```

💡 *Closures are used everywhere in JavaScript: event handlers, callbacks, module patterns, memoisation. The closure "closes over" the variable — keeps it alive in memory as long as the closure exists.*

Q7. What is a prototype?

Every JavaScript object has an internal `[[Prototype]]` link to another object (its prototype).

When you access a property/method, JS looks in the object first, then in its prototype, then prototype's prototype — this is the prototype chain.

All objects ultimately link back to `Object.prototype` (which links to null).

▶ Prototype chain:

```
// Every function has a .prototype property
function Animal(name) { this.name = name; }
Animal.prototype.speak = function() { // added to prototype – SHARED
  console.log(`${this.name} makes a sound`);
};

const dog = new Animal('Rex');
dog.speak(); // Rex makes a sound (found on prototype)
dog.hasOwnProperty('name'); // true (own property)
dog.hasOwnProperty('speak'); // false (inherited from prototype)

// Prototype chain: dog → Animal.prototype → Object.prototype → null
console.log(Object.getPrototypeOf(dog) === Animal.prototype); // true

// ES6 class is syntactic sugar – same prototype chain under the hood
class Dog extends Animal {
  bark() { console.log('Woof!'); }
}
const rex = new Dog('Rex');
rex.speak(); // uses Animal.prototype.speak (inherited)
rex.bark(); // uses Dog.prototype.bark
```

Q8. What is spread operator?

Spread (...) expands an iterable (array, string, object) into individual elements.

Uses: merge arrays, clone arrays/objects, spread function arguments, merge objects.

▶ Spread operator examples:

```
// Array merge
const arr1 = [1, 2, 3];
const arr2 = [4, 5, 6];
const merged = [...arr1, ...arr2]; // [1,2,3,4,5,6]

// Array clone (shallow)
const original = [1, 2, 3];
const clone = [...original]; // new array – not same reference
clone.push(4);
console.log(original); // [1,2,3] – untouched
```

```
// Object merge (later properties override earlier ones)
const defaults = { theme: 'dark', lang: 'en', size: 'md' };
const userPrefs = { theme: 'light', fontSize: 14 };
const settings = { ...defaults, ...userPrefs };
// { theme: 'light', lang: 'en', size: 'md', fontSize: 14 }

// Function call – spread array as arguments
function add(a, b, c) { return a + b + c; }
const nums = [1, 2, 3];
console.log(add(...nums)); // 6 (same as add(1,2,3))
```

Q9. What is rest operator?

Rest (...) collects remaining arguments into an array. Same syntax as spread but opposite purpose.

Spread: expands. Rest: collects.

Most common use: function parameters where you don't know how many arguments will be passed.

▶ Rest operator examples:

```
// Rest parameter – must be LAST parameter
function sum(first, second, ...others) {
  console.log(first); // 1
  console.log(second); // 2
  console.log(others); // [3, 4, 5]
  return first + second + others.reduce((acc, n) => acc + n, 0);
}
console.log(sum(1, 2, 3, 4, 5)); // 15

// Rest in destructuring – collect remaining array elements
const [head, ...tail] = [1, 2, 3, 4, 5];
console.log(head); // 1
console.log(tail); // [2, 3, 4, 5]

// Rest in object destructuring
const { name, ...rest } = { name: 'Vamshi', role: 'Dev', city: 'Ongole' };
console.log(name); // Vamshi
console.log(rest); // { role: 'Dev', city: 'Ongole' }
```

💡 *Spread = fan out (one → many). Rest = gather in (many → one). Same ... symbol, context determines which behaviour applies.*

Q10. Difference between spread and rest operator

Spread: used when PASSING or USING values — expands collection into individuals.

Rest: used when RECEIVING values — gathers individuals into a collection.

Same ... syntax — the difference is WHERE it appears:

In function CALL / array/object literal = spread.

In function DEFINITION parameters / destructuring = rest.

Q11. splice vs slice

slice: non-mutating — returns a new sub-array. Original unchanged. arr.slice(start, end) — end is exclusive.

splice: MUTATING — modifies original array. Can add, remove, or replace elements. Returns removed elements.

▶ slice vs splice side by side:

```
const fruits = ['apple', 'banana', 'cherry', 'date', 'elderberry'];

// — SLICE: read-only sub-array —————
const sliced = fruits.slice(1, 3); // start=1, end=3 (exclusive)
console.log(sliced); // ['banana', 'cherry']
console.log(fruits); // ['apple', 'banana', 'cherry', 'date', 'elderberry'] – UNCHANGED

// Negative index: count from end
const last2 = fruits.slice(-2); // last 2 elements
console.log(last2); // ['date', 'elderberry']
```

```
// — SPLICE: modify original array —————
const arr = ['a', 'b', 'c', 'd'];

// Remove 2 elements starting at index 1
const removed = arr.splice(1, 2); // start=1, deleteCount=2
console.log(removed); // ['b', 'c'] (returned removed elements)
console.log(arr); // ['a', 'd'] (ORIGINAL MODIFIED)

// Insert without removing (deleteCount=0)
arr.splice(1, 0, 'X', 'Y');
console.log(arr); // ['a', 'X', 'Y', 'd']

// Replace 1 element
arr.splice(1, 1, 'Z');
console.log(arr); // ['a', 'Z', 'Y', 'd']
```

Q12. What is arrow function?

Arrow functions (=>) are a concise syntax for functions.

Key differences from regular functions:

1. No own "this" — inherits from surrounding (lexical) scope.
2. No "arguments" object.
3. Cannot be used as constructor (no "new").
4. Cannot be used as methods when "this" context needed.

▶ Arrow function vs regular function:

```
// Regular function
function add(a, b) { return a + b; }

// Arrow function equivalents
const add1 = (a, b) => a + b; // implicit return when no braces
const add2 = (a, b) => { return a + b; }; // explicit return with braces
const double = n => n * 2; // single param — parens optional
const greet = () => 'Hello!'; // no params

// KEY DIFFERENCE: 'this' binding
class Timer {
  constructor() { this.seconds = 0; }

  // Regular function — 'this' depends on how function is called
  startRegular() {
    setInterval(function() {
      this.seconds++; // 'this' is undefined (strict) or window — BUG!
    }, 1000);
  }

  // Arrow function — 'this' = Timer instance (lexical 'this')
  startArrow() {
    setInterval(() => {
      this.seconds++; // 'this' IS the Timer instance — CORRECT
    }, 1000);
  }
}
```

🔗 The "this" difference is the most important. Arrow functions are perfect for callbacks inside class methods because they capture the class instance's "this" automatically.

Q13. What is anonymous function?

A function with no name. Used as callbacks, immediately invoked, or assigned to variables.

Cannot be called before declaration (unlike named function declarations).

▶ Anonymous function uses:

```
// Assigned to variable
const multiply = function(a, b) { return a * b; }; // anonymous function expression
```

```
// Arrow function (also anonymous by default)
const divide = (a, b) => a / b;

// As callback (most common use)
[1, 2, 3].forEach(function(n) { console.log(n); }); // anonymous callback
[1, 2, 3].forEach(n => console.log(n)); // arrow anonymous

// IIFE - Immediately Invoked Function Expression (runs immediately, scope isolation)
(function() {
    const secret = 'only here';
    console.log('IIFE ran');
})(); // the () at end immediately invokes it
// console.log(secret); // ReferenceError - secret is isolated
```

Q14. What is array and its types?

Standard Array: ordered, any types mixed, resizable, zero-indexed.

Typed Arrays: Int8Array, Uint8Array, Float32Array, Float64Array — fixed-size, single type, direct memory buffer. Used for WebGL, audio/video processing, binary data.

Array-like: NodeList, arguments, HTMLCollection — have length and indices but not Array methods. Convert with Array.from().

▣ Array types:

```
// Standard array
const mixed = [1, 'hello', true, null, { id: 1 }];

// Typed array - for performance-critical binary data
const buffer = new Float64Array(3); // 3 doubles
buffer[0] = 3.14; buffer[1] = 2.71; buffer[2] = 1.41;
console.log(buffer); // Float64Array [3.14, 2.71, 1.41]

// Convert array-like to real array
const nodeList = document.querySelectorAll('div'); // NodeList
const realArr = Array.from(nodeList); // now has .map, .filter etc.

// Useful array methods
const nums = [1,2,3,4,5];
console.log(nums.map(n => n*2)); // [2,4,6,8,10]
console.log(nums.filter(n => n % 2 === 0)); // [2,4]
console.log(nums.reduce((sum,n) => sum+n, 0)); // 15
console.log(nums.find(n => n > 3)); // 4
console.log(nums.every(n => n > 0)); // true
console.log(nums.some(n => n > 4)); // true
```

Q15. How to flatten multi-dimensional array?

▣ flat() and reduce approaches:

```
const nested = [1, [2, 3], [4, [5, 6]], [7, [8, [9]]]];

// flat(n) - flatten n levels deep
console.log(nested.flat(1)); // [1,2,3,4,[5,6],7,[8,[9]]] one level
console.log(nested.flat(2)); // [1,2,3,4,5,6,7,[9]] two levels
console.log(nested.flat(Infinity)); // [1,2,3,4,5,6,7,8,9] ALL levels

// Manual deep flatten using recursion
function deepFlatten(arr) {
    return arr.reduce((flat, item) =>
        Array.isArray(item) ? [...flat, ...deepFlatten(item)] : [...flat, item],
        []);
}
console.log(deepFlatten(nested)); // [1,2,3,4,5,6,7,8,9]
```

Q16. = vs == vs === — name them and explain

= (Assignment operator): assigns a value to a variable. x = 5.

`==` (Loose equality / Abstract equality): compares VALUES after TYPE COERCION. `5 == "5"` is TRUE.
`===` (Strict equality): compares VALUE AND TYPE. No coercion. `5 === "5"` is FALSE. ALWAYS use `===`.

► Equality operator comparison:

```
// Assignment
let x = 5; // x now holds 5

// Loose equality == (coerces types - AVOID)
console.log(5 == "5"); // true - string coerced to number
console.log(0 == false); // true - false coerced to 0
console.log(0 == ""); // true - both coerce to 0
console.log(null == undefined); // true - special rule
console.log([] == false); // true - [] → 0, false → 0

// Strict equality === (no coercion - USE THIS)
console.log(5 === "5"); // false - different types
console.log(0 === false); // false - different types
console.log(null === undefined); // false - different types

// Common bug: if (x = 5) - ASSIGNMENT inside if!
// if (x == 5) - loose comparison
// if (x === 5) - CORRECT strict comparison
```

🔗 Always use `===` in interviews. The rule "use strict equality" is one of the most universally agreed best practices in JavaScript.

Q17. Output questions: `console.log(true+1)`, `console.log(1+'2')`, `console.log('2'+1)`

These test JavaScript's type coercion rules:

`true + 1`: boolean coerced to number (`true=1`), result is 2.

`1 + "2"`: number + string triggers string concatenation, result is "12".

`"2" + 1`: string + number triggers string concatenation, result is "21".

► Type coercion output:

```
// + operator: if EITHER operand is string → string concatenation
// + operator: if BOTH are numbers (or booleans) → arithmetic addition

console.log(true + 1); // 2 (true → 1, then 1 + 1 = 2)
console.log(false + 1); // 1 (false → 0, then 0 + 1 = 1)
console.log(1 + '2'); // "12" (number + string = concatenation)
console.log('2' + 1); // "21" (string + number = concatenation)
console.log('2' - 1); // 1 (- forces numeric, '2' → 2, 2 - 1 = 1)
console.log('5' * 2); // 10 (* forces numeric)
console.log('5'); // 5 (unary + converts string to number)

// Virtusa question answers:
let a = '3' + 3 + 3; // "333" left-to-right: '3'+3='33', '33'+3='333'
let b = 3 + '3' + 3; // "333" 3+'3'='33', '33'+3='333'
let c = 3 + 3 + '3'; // "63" 3+3=6 (both numbers), 6+'3'='63'
let d = '3' + 3 - 3; // 30 '3'+3='33', '33'-3=30 (- forces numeric)
let e = 3 + 3 + 3; // 9 all numbers
console.log(a, b, c, d, e);
```

Q18. Output: `console.log(a)`; `console.log(b)`; `console.log(c)`; `var a`; `let b`;

► Hoisting + TDZ output question:

```
// let c; // not declared - will throw ReferenceError
console.log(a); // undefined - var 'a' is hoisted (declaration moved up)
console.log(b); // ReferenceError: Cannot access 'b' before initialization
// console.log(c); // ReferenceError: c is not defined (not declared at all)

var a; // declaration (already hoisted above)
let b; // declaration - TDZ ends here

// If we also add: let c; at top
// After declaring:
```



```
let c2;
console.log(c2);           // undefined – declared but not initialised
```

Q19. How to pass data from parent to child component (React)?

▶ Props passing:

```
// PARENT component
function ParentComponent() {
  const [userName, setUserName] = React.useState('Vamshi');
  const [score, setScore] = React.useState(95);

  function handleUpdate(newName) {
    setUserName(newName); // update parent state
  }

  return (
    <div>
      {/* Pass data DOWN via props (parent → child) */}
      <ChildComponent
        name={userName}           // primitive prop
        score={score}            // number prop
        onChange={handleUpdate}  // function prop (callback for child→parent)
      />
    </div>
  );
}

// CHILD component – receives via props parameter
function ChildComponent({ name, score, onChange }) {
  return (
    <div>
      <h2>Name: {name}</h2>
      <p>Score: {score}</p>
      {/* Child sends data UP by calling the callback prop */}
      <button onClick={() => onChange('Reddy')}>
        Change Name
      </button>
    </div>
  );
}
```

🔗 Props are READ-ONLY in the child — child should never modify props directly. To update parent state from child, pass a callback function as a prop and call it from the child.

Q20. What is state and props?

STATE: internal data OWNED by the component. Mutable via `useState()`. Triggers re-render when updated. Private to the component (unless shared via props).

PROPS: data RECEIVED from parent. Read-only in child. Component re-renders when props change. Used to pass data and callbacks down the tree.

▶ State vs Props:

```
function Counter({ initialCount, onReset }) {
  // STATE: owned by this component, can change
  const [count, setCount] = React.useState(initialCount); // initialCount is prop, count is state

  return (
    <div>
      <p>Count: {count}</p>
      {/* Update OWN state */}
      <button onClick={() => setCount(c => c + 1)}>+1</button>
      {/* Call PARENT callback to inform parent */}
      <button onClick={onReset}>Reset</button>
    </div>
  );
}
```

```
// Parent
function App() {
  const [resetCount, setResetCount] = React.useState(0);
  return (
    <Counter
      initialCount={0} // PROP - passed in
      onReset={() => setResetCount(n => n+1)} // PROP - callback
    />
  );
}
```

Q21. If UI page is loading very slowly — steps to optimize?

DIAGNOSE first, then fix:

1. Chrome DevTools → Network tab: find large JS bundles, slow API calls, too many requests.
2. Lighthouse audit: get performance score with specific recommendations.
3. React DevTools Profiler: find components re-rendering unnecessarily.

FIXES:

- Code splitting: React.lazy() + Suspense for large components.
- Image optimisation: WebP format, lazy loading, correct sizes.
- Bundle size: tree shaking, remove unused dependencies.
- Memoisation: React.memo, useMemo, useCallback to reduce re-renders.
- Virtual scrolling for long lists (react-window, react-virtual).
- Move API calls to useSWR/React Query for caching.
- CDN for static assets.

Q22. What is code splitting?

Code splitting divides your JavaScript bundle into smaller chunks that are loaded ON DEMAND instead of all upfront.

Benefit: initial page load downloads only the code needed for the first screen — faster first paint.

Route-based is most effective: each page's code is a separate chunk, loaded when user navigates there.

▶ Code splitting with React.lazy:

```
import React, { Suspense, lazy } from 'react';

// Without code splitting: ALL these components are in the same bundle
// import AdminDashboard from './AdminDashboard'; // 200KB
// import Reports from './Reports'; // 150KB
// import Analytics from './Analytics'; // 100KB
// → Initial bundle: 450KB — user must download ALL even if they never visit admin

// WITH code splitting: each loaded only when route is visited
const AdminDashboard = lazy(() => import('./AdminDashboard')); // separate chunk
const Reports = lazy(() => import('./Reports'));
const Analytics = lazy(() => import('./Analytics'));

function App() {
  return (
    <Router>
      {/* Suspense shows fallback while chunk is loading */}
      <Suspense fallback={<div>Loading...</div>}>
        <Routes>
          <Route path="/admin" element={<AdminDashboard />} />
          <Route path="/reports" element={<Reports />} />
          <Route path="/analytics" element={<Analytics />} />
        </Routes>
      </Suspense>
    </Router>
  );
}

// Now initial bundle is just App + Router. Admin code downloaded only when user visits /admin
```

Q23. What is Redux?

Redux: predictable state container for JavaScript applications.

Core concepts:

STORE: single source of truth — the entire app state in one object.

ACTION: plain object describing what happened: { type: "ADD_ITEM", payload: item }.

REDUCER: pure function (currentState, action) → newState. Must be side-effect free.

DISPATCH: send an action to the store.

SELECTOR: function to read specific part of state.

Redux Toolkit (RTK): modern Redux — less boilerplate, includes createSlice, createAsyncThunk.

▶ Redux Toolkit example:

```
// cartSlice.js
import { createSlice } from '@reduxjs/toolkit';

const cartSlice = createSlice({
  name: 'cart',
  initialState: { items: [], total: 0 },
  reducers: {
    // createSlice uses Immer — can "mutate" state directly (actually immutable under hood)
    addItem: (state, action) => {
      state.items.push(action.payload);
      state.total += action.payload.price;
    },
    removeItem: (state, action) => {
      state.items = state.items.filter(i => i.id !== action.payload);
    },
    clearCart: (state) => {
      state.items = [];
      state.total = 0;
    }
  }
});

export const { addItem, removeItem, clearCart } = cartSlice.actions;
export const selectCartItems = state => state.cart.items;
export const selectCartTotal = state => state.cart.total;
export default cartSlice.reducer;

// Component usage
function CartButton({ product }) {
  const dispatch = useDispatch();
  const total = useSelector(selectCartTotal); // reads from store

  return (
    <div>
      <span>Total: {total}</span>
      <button onClick={() => dispatch(addItem(product))}>Add to Cart</button>
    </div>
  );
}
```

Q24. What is Context API?

React Context: built-in state sharing mechanism for passing data through the component tree without prop drilling.

Create context → wrap tree with Provider → consume with useContext().

When to use: theme, auth user, language settings — data that many components need but rarely changes.

When NOT to use: frequently changing state (causes all consumers to re-render) — use Redux/Zustand instead.

▶ Context API example:

```
// 1. Create the context
const ThemeContext = React.createContext({ theme: 'light', toggle: () => {} });
```

```
// 2. Provider wraps the tree
function ThemeProvider({ children }) {
  const [theme, setTheme] = React.useState('light');

  const toggle = () => setTheme(t => t === 'light' ? 'dark' : 'light');

  return (
    // Any component inside can access theme and toggle
    <ThemeContext.Provider value={{ theme, toggle }}>
      {children}
    </ThemeContext.Provider>
  );
}

// 3. Consumer — anywhere in the tree, no prop drilling
function Header() {
  const { theme, toggle } = React.useContext(ThemeContext);
  return (
    <header style={{ background: theme === 'dark' ? '#333' : '#fff' }}>
      <button onClick={toggle}>Toggle Theme</button>
    </header>
  );
}

// App
function App() {
  return (
    <ThemeProvider>
      <Header />           {/* can access theme */}
      <MainContent />      {/* can access theme */}
      <Footer />           {/* can access theme */}
    </ThemeProvider>
  );
}
```

Q25. How to increase Angular application performance?

1. OnPush change detection: only check component when @Input reference changes — reduces CD cycles by 80%+.
2. trackBy in *ngFor: prevents DOM recreation when list data changes.
3. Async pipe: auto-subscribes/unsubscribes, no manual cleanup.
4. Lazy loading routes: load feature modules only when navigated to.
5. Virtual scrolling (CDK): render only visible rows for long lists.
6. AOT compilation: compile at build time — faster runtime.
7. Tree-shaking: remove unused code at build.
8. Memoize expensive pipe calculations (pure pipes).
9. Avoid function calls in templates — use getters or pre-compute.
10. Bundle budgets: set max sizes in angular.json, fail build if exceeded.

Q26. If there is a bulk upload case for photos/videos — what is your approach?

1. CHUNKED UPLOAD: split large files into 5-10MB chunks. Upload each chunk independently. Reassemble on server. If one chunk fails, retry only that chunk (not entire file).
2. PRESIGNED URLs: client requests presigned S3/Blob URL from API → client uploads DIRECTLY to storage (bypasses your server, saves bandwidth).
3. BACKGROUND PROCESSING: after upload, queue a job (Azure Queue/Service Bus) for video transcoding, image compression, thumbnail generation.
4. PROGRESS TRACKING: use SignalR or polling to report progress to frontend.
5. RATE LIMITING: restrict concurrent uploads per user.
6. VALIDATION: file type, file size, virus scan before processing.

Q27. Angular application is slow — how to optimise?

Same as Q25 above with additional:

- Profile with Chrome DevTools + Angular DevTools: find slow components.
- Check *ngFor loops for missing trackBy.
- Check for expensive operations in template (function calls, unoptimised pipes).
- Enable production build (ng build --configuration production) — has AOT, minification, tree-shaking.
- Implement virtual scrolling for lists > 100 items.
- Reduce bundle size: analyze with source-map-explorer or webpack-bundle-analyzer.

SECTION 13: SYSTEM DESIGN & ARCHITECTURE (27 Questions)

Q1. K8s design strategy — VNet, node pools, microservices

DESIGN:

System node pool: 2-3 nodes (D2s_v3) — runs K8s system pods.

App node pool(s): auto-scale 3-20 nodes — runs your services.

One namespace per team/environment.

One Deployment + Service per microservice.

Ingress controller (nginx or AGIC) as single entry point for external traffic.

All in private VNet — no public IPs on pods/nodes.

Managed Identity for pod access to Key Vault, ACR, Storage.

Q2. One pod — multiple containers of different types?

Yes. Pod can have: Main container (your app), Sidecar container (log forwarder, service mesh proxy like Envoy), Init container (runs to completion BEFORE main starts — e.g., wait for DB, run migrations). All share same network namespace and can communicate via localhost.

Q3. Ingress routes traffic — explain the full flow

Client → DNS → Ingress Controller (LoadBalancer IP) → Ingress Resource (path rules) → Service (virtual IP, load balances to pods) → Pod (actual container). Ingress resource defines: host: api.myapp.com, path: /orders → service: order-svc:80.

Q4. How does a pod connect to Azure Storage / Key Vault?

Workload Identity (recommended): annotate K8s ServiceAccount with Azure Managed Identity client ID → Azure Identity Webhook mutates pod with environment variables → DefaultAzureCredential in code picks it up → authenticates to Azure AD → accesses Storage/KeyVault without any secrets in pod spec.

Q5. Are SOLID principles applicable to microservices?

Yes — applied at service level:

S: each service has one bounded context.

O: add new services without changing existing.

L: services expose contracts that substitutable implementations honour.

I: small, focused API contracts per service.

D: services depend on contracts (API specs, message schemas), not each other's internals.

Q6. Sidecar design pattern?

A sidecar container runs in the SAME pod as the main application container. Handles cross-cutting concerns without changing the main app:

- Logging sidecar: collects and forwards logs (Fluentd, Filebeat).
- Service mesh proxy: Envoy (Istio) handles retries, circuit breaking, mTLS.
- Secrets injector: Vault sidecar injects secrets as files.
- Health check sidecar: external health monitoring.

Main app stays clean — sidecar augments it.

Q7. Docker containers — how to create a container?

1. Write Dockerfile.
2. `docker build -t myapp:1.0 .` (build image from Dockerfile).
3. `docker run -d -p 8080:80 --name myapp myapp:1.0` (create and start container).
4. `docker logs myapp` (view logs).
5. `docker exec -it myapp /bin/bash` (shell into container).
6. `docker stop myapp && docker rm myapp` (stop and remove).

Q8. How to select a Docker base image?

Criteria:

1. Official/verified publisher images (microsoft, node, python) — security maintained.
2. Minimal size: use `-slim` or `-alpine` variants. Smaller = fewer CVEs, faster pull.
3. Specific tag (not `:latest`) — reproducible builds. `:latest` can change unexpectedly.
4. Right runtime: `mcr.microsoft.com/dotnet/aspnet:8.0` (runtime only, smaller than SDK image).
5. CVE scan: Docker Scout, Trivy, Snyk to check for vulnerabilities.

Q9. How to approach project breakdown and design?

- Step 1: Understand domain — talk to business, identify bounded contexts.
- Step 2: Identify subdomains — core, supporting, generic.
- Step 3: Define services — one per bounded context ideally.
- Step 4: Define APIs/contracts — OpenAPI spec first (spec-driven development).
- Step 5: Data model — what DB per service, relational vs NoSQL.
- Step 6: Communication patterns — sync (HTTP/gRPC) vs async (events).
- Step 7: Infrastructure — deployment, scaling, observability.

Q10. How to define bounded context when there are overlaps?

Use Anti-Corruption Layer (ACL): translation layer between contexts — each context has its own model, ACL translates.

Context Map: document relationships — Conformist (downstream accepts upstream model), Customer-Supplier (negotiate), Shared Kernel (agreed shared model).

For GENUINE overlaps: use domain events to sync data across contexts — eventual consistency.

Q11. Event sourcing and eventual consistency with database

Event sourcing: store events not state. Read model rebuilt from events.

Eventual consistency: write happens to write DB first (event stored), read model updates asynchronously (via event subscriber). For a brief moment read model is stale — this is acceptable for most use cases.

Q12. Implement SAGA pattern practically

ORCHESTRATION SAGA (recommended for clarity):

Create OrderSaga class in OrderService. Steps: ReserveInventory → ProcessPayment → SendConfirmation. On any failure: execute compensating calls in reverse (RefundPayment → ReleaseInventory → CancelOrder). Use Polly for retries in each step. Store saga state in DB for recovery after crash.

Q13. SAGA for distributed transactions

Replaces 2-phase commit (which is blocking and doesn't work well across services). Each participant can roll back via compensating transaction. No distributed lock needed. Trade-off: eventual consistency instead of ACID.

Q14. How many services — is SAGA separate?

SAGA orchestrator logic lives INSIDE the service that initiates the workflow (e.g. inside OrderService). It is a class, not a separate service. It coordinates calls to OTHER services (PaymentService, InventoryService) via HTTP or events.

Q15. Where does saga sit?

Inside the initiating service. OrderService.PlaceOrder() method IS the orchestrator — it calls payment, inventory, notification in sequence and handles compensations.

Q16. Where is the saga logic implemented?

In an OrderSaga class inside OrderService project. Methods: ExecuteAsync() for forward path, CompensateAsync() for rollback. Saga state persisted in a SagaState table.

Q17. Inter-service SAGA — who coordinates?

The ORCHESTRATOR service (producer/initiating service) takes responsibility. Individual services (PaymentService, InventoryService) are stateless participants — they just do what they are asked and emit events.

Q18. Consumer or producer responsible for SAGA?

ORCHESTRATOR (typically the producer) is responsible. In CHOREOGRAPHY SAGA, each service reacts to events — distributed responsibility. In ORCHESTRATION SAGA, one coordinator controls the flow — clearer responsibility.

Q19. Explain event sourcing

Every state change stored as an immutable event in an append-only log.

Examples: OrderCreated, PaymentProcessed, ItemShipped.

State rebuilt by replaying events: sum all "balance added" and "balance deducted" events = current balance.

Benefits: complete audit trail, time-travel debugging, replay for new read-models, natural fit for CQRS.

Q20. .NET Web API connect to private Azure resources

Assign System Managed Identity to App Service. Grant RBAC roles (Key Vault Secrets User, SQL Server contributor). Enable private endpoints on SQL/Key Vault (private IP inside VNet). Use App Service VNet integration to access the VNet. Code: DefaultAzureCredential() handles all auth — no secrets.

Q21. Manage SQL connections efficiently under high load

Built-in connection pooling in SqlConnection (min/max pool size in connection string). Async queries (await ExecuteAsync) — threads not blocked while waiting. CQRS: route read queries to read replica. Retry with backoff on transient failures (Polly). Monitor connection pool saturation via App Insights.

Q22. Connection pool limit — serve every request without rejecting

Queue requests using SemaphoreSlim to limit concurrent DB calls. Excess requests wait in the semaphore queue instead of failing. Additionally: scale horizontally (more app instances), increase pool size, use read replica for read traffic.

Q23. CQRS solves connection pressure

Write DB handles only mutations (lower volume). Read replicas handle queries (higher volume). Each side sized independently. Total connections split across two servers — effectively doubles connection capacity.

Q24. Azure OpenAI integration — use case

RAG (Retrieval-Augmented Generation): embed documents → store vectors in Azure AI Search → user query → embed query → similarity search → retrieve relevant docs → augment GPT-4 prompt → get accurate, grounded response. Used for: internal knowledge base chat, document Q&A, code generation, content summarisation.

Q25. Spec-driven development?

API-first approach: write OpenAPI spec (Swagger YAML) BEFORE implementing. Generate server stubs and client SDKs from spec. Frontend and backend teams can work in parallel — frontend uses mock server from spec. Benefits: clear contract, early agreement, auto-generated documentation, type-safe clients.

Q26. TinyURL design on-premise

Short code: Base62 encode auto-increment ID (a-zA-Z0-9). DB table: id, short_code, long_url, customer_id, clicks, created_at. Redirect service: GET /{code} → DB lookup → 301 Redirect → increment click counter (async). Cache popular URLs in Redis (90% of traffic to top 10% URLs). Rate limit per customer API key. Multiple nodes behind load balancer for high availability.

Q27. RAG architecture on-premise

Documents → chunk (512 tokens each) → embed with sentence-transformers model (all-MiniLM-L6-v2) → store vector + metadata in pgvector (PostgreSQL extension) or Qdrant. Query time: embed user question → cosine similarity search → top K chunks → build prompt → LLM (Llama 3 or Mistral via Ollama on GPU server) → response. All on-premise — no external API calls.

SECTION 14: BEHAVIORAL & HR QUESTIONS (89 Questions — Key Answers)

Q1. Tell me about yourself

Structure (2 min max):

"I am a .NET Full Stack Developer with X years of experience. Currently at Qatar Airways where I build customer-facing travel management systems using Angular, .NET Core, Azure, and SQL Server.

I have strong expertise in REST API design, microservices architecture, and Azure cloud services.

I am seeking a Senior/Lead role where I can contribute to architectural decisions and mentor junior developers."

Q2. Describe current project and architecture

Prepare: Business purpose → Tech stack → Architecture pattern (monolith/microservices) → Your specific modules → Key design decisions → Challenges you solved.

Be honest. Interviewers probe deeply — inconsistencies are noticed immediately.

Q3. Walk through projects in CV

For each: What problem it solved → Your role → Tech stack and WHY chosen → Key challenge you personally solved → Outcome/impact.

Quantify when possible: "Reduced API response time by 40% by implementing caching."

Q4. Design patterns used in your project

Have 3-4 patterns ready with REAL examples: "I used Repository pattern to abstract EF Core from the service layer, which allowed us to mock the data layer in unit tests. I used Factory pattern for payment gateway selection — new gateways added without touching existing code."

Q5. How many microservices — what does each handle?

Have specific answer for YOUR project. E.g., "We have 5 services: Auth (JWT issuance/validation), Booking (flight selection/hold), Payment (charge/refund), Notification (email/SMS), and Reporting (analytics queries). Each owns its own SQL DB."

Q6. Does each microservice have its own database?

Correct architectural answer: "Yes, database-per-service for true independence." If shared in your project: "We share the DB for now due to legacy constraints, but we have clear schema ownership per service and no cross-schema JOINS."

Q7. Why are you directly querying another service's DB?

This exposes anti-pattern. Correct answer: "You are right — that is an architectural debt we have identified. The proper approach is to either call the service's API endpoint or subscribe to domain events it publishes. We plan to address this in the next sprint."

Q8. Redis cache for master data instead of cross-DB joins?

"Yes, that would be a better approach. The owning service publishes events when master data changes. Dependent services cache it in Redis with event-based invalidation — eventual consistency acceptable for reference data like airport codes, aircraft types."

Q9. Which message broker for inter-service communication?

Be specific to your project. "We use Azure Service Bus for guaranteed delivery and dead-letter queue support. For high-throughput analytics events we use Azure Event Hub."

Q10. Authentication mechanism?

"JWT bearer tokens issued on login. Tokens contain role claims. Every API validates signature and expiry via AddJwtBearer middleware. Service-to-service calls use Client Credentials flow (OAuth 2.0) with Managed Identity."

Q11. Walk through auth flow

User submits credentials → Auth service validates → Issues JWT (signed with RS256 private key) → Client stores token → Every API request includes Bearer token → Middleware validates signature against public key → Claims extracted → Authorization policy checked.

Q12. API Gateway for cross-cutting concerns?

If yes: "Azure APIM in front of all services. Handles JWT validation, rate limiting, IP filtering, request logging at gateway level — services focus only on business logic." If no: "We handle auth in each service — acknowledged limitation, plan to add APIM."

Q13. DDOS attack — how to mitigate?

Azure Front Door WAF: rate limiting, IP reputation filtering, geo-blocking. Azure DDoS Protection Standard. API rate limiting at APIM. CDN absorbs volumetric attacks. Circuit breaker in services (stop processing when overloaded).

Q14. AKS deployment — how K8s works

Pods: smallest unit (1+ containers). Node: VM running pods. Deployment: desired state for pods (replicas, image). Service: stable network endpoint to pods. Ingress: HTTP routing to services. HPA: auto-scales pods on CPU/memory. Cluster Autoscaler: auto-scales nodes.

Q15. Service discovery in K8s?

K8s DNS: each Service gets a DNS name: servicename.namespace.svc.cluster.local. Pods call each other by service name. No hardcoded IPs. Example: `http://payment-svc/api/charge` (payment-svc resolves to Service's ClusterIP which load-balances to pods).

Q16. OAuth 2.0 Authorization Code Flow vs PKCE?

Auth Code Flow: for server-side apps with secure client secret. PKCE (Proof Key for Code Exchange): for SPAs and mobile apps where client secret cannot be kept secret. PKCE uses `code_verifier` + `code_challenge` instead of client secret. Always use PKCE for public clients.

Q17. Minimum config to register app in Azure AD?

App name, supported account types (single tenant / multi-tenant), redirect URI. Optionally: app roles, API permissions, certificates. Portal generates `client_id` and `tenant_id` automatically.

Q18. 3 parts of JWT and role of each?

Header: algorithm (RS256/HS256) and token type (JWT). Payload: claims (`sub`=user ID, `iat`=issued at, `exp`=expiry, `roles`=["Admin"]). Signature: header+payload hashed with private key. Recipient validates signature with public key — ensures nobody tampered.

Q19. Validate JWT from API side — what claims to check?

`iss` (issuer) — must match expected issuer URL. `aud` (audience) — must match your API. `exp` (expiry) — must be in future. `nbf` (not before) — must be in past. `iat` (issued at) — must be reasonable. Role/scope claims for authorization. Signature — validates integrity.

Q20. What is a resource leak?

Failing to release unmanaged resources (file handles, DB connections, HTTP clients) after use. Causes: running out of handles, memory growth, connection pool exhaustion. Prevention: `IDisposable` pattern + `using` statement, `IHttpClientFactory` (not `new HttpClient()`), connection pooling.

Q21. Why use `using` keyword in C#?

Ensures `IDisposable.Dispose()` is called even if exception occurs — equivalent to `try-finally`. Prevents resource leaks. Works with anything implementing `IDisposable`: `DbContext`, `SqlConnection`, `FileStream`, `HttpClient`. C# 8+ `using` declaration: `using var conn = new SqlConnection(cs);` disposed at end of scope.

Q22. What is a race condition in banking transactions?

Two concurrent transactions read same balance (1000), both add 500, both write 1500 — result should be 2000 but is 1500. Fix: DB-level: use transactions with `SERIALIZABLE` isolation or row-level locking. App-level: optimistic concurrency (EF Core row version), pessimistic locking, or queue-based serialisation.

Q23. Why use database transactions?

ACID guarantees: Atomicity (all or nothing — debit AND credit), Consistency (business rules enforced), Isolation (concurrent transactions don't interfere), Durability (committed data survives crash). Essential for any multi-step operation affecting multiple rows or tables.

Q24. Show total balance per customer across accounts

SQL: `SELECT c.CustomerId, c.Name, SUM(a.Balance) AS TotalBalance FROM Customers c JOIN Accounts a ON c.CustomerId = a.CustomerId GROUP BY c.CustomerId, c.Name;`

Q25. Optimise query for latest transaction per account

CTE with `ROW_NUMBER()`: `WITH Latest AS (SELECT AccountId, Amount, TxDate, ROW_NUMBER() OVER (PARTITION BY AccountId ORDER BY TxDate DESC) AS RN FROM Transactions) SELECT * FROM Latest WHERE RN = 1;` — indexes on `(AccountId, TxDate DESC)` are essential.

Q26. Tell me about a conflict in your team

STAR format: Situation (what was the context), Task (what was your role), Action (what you specifically did), Result (outcome). Example: "Two team members disagreed on architecture. I facilitated a technical discussion, presented pros/cons of each approach with benchmarks, and we reached consensus on a hybrid solution."

Q27. Manage multiple projects at same time?

Prioritise by business impact and deadline. Communicate blockers early. Time-box deep work per project. Use sprint planning to balance. Delegate where possible. "I use a daily priority list — high-impact items done before 10 AM."

Q28. Team member not performing?

First: 1:1 conversation to understand root cause (personal issues? unclear requirements? skill gap? motivation?). Then: provide specific feedback, offer mentoring/pairing, set clear expectations. Escalate to manager only after giving fair chance to improve. Document conversations.

Q29. Brief technical exposure and recent projects

2-minute structured answer: years of experience + core stack → most complex project (name, problem, your contribution) → key technical achievement → what you're doing now. Practice this — it sets the tone for the entire interview.

Q30. Could you replicate master data to avoid cross-DB joins?

Yes — this is the correct microservices pattern. Copy reference data into each service's own DB via event sourcing. Downstream service subscribes to "MasterDataUpdated" events and updates its local copy. Accepts eventual consistency for reference data.

Key Behavioral Questions (Continued)

Q31. Decision making / trade-offs

Framework: gather requirements → identify constraints → list options → evaluate against criteria (cost, time, complexity, scalability) → make recommendation with justification → get buy-in. Show you can articulate the trade-off clearly: "Option A is faster but harder to maintain. Given our deadline, I recommend A with a plan to refactor in Q3."

Q32. Scalability thinking

Think in terms of: horizontal scaling (more instances), caching (Redis, CDN), async processing (queues), DB read replicas, stateless design. Always ask "what happens when traffic is 10x?"

Q33. Debugging mindset

Reproduce locally → isolate → check logs and metrics → form hypothesis → test → fix → add test to prevent regression. For production: correlation ID tracing, Application Insights, check recent deployments.

Q34. Experience with RESTful APIs

Mention: designing clean resource URLs, proper HTTP verbs, status codes, pagination, versioning, input validation, JWT auth, Swagger documentation, integration testing.

Q35. Ensure quality and reliability of APIs

Unit tests (xUnit, Moq), integration tests (TestContainers with real DB), load testing (k6/NBomber), code coverage gates (>80%), peer code review, API contract tests, health check endpoints, alerting on error rate.

Q36. Retry policies for transient failures — Polly

Polly: .NET resilience library. HttpClient decorated with retry policy: retry 3 times with exponential backoff for 5xx/timeout. Circuit breaker: after 5 failures in 30s, open circuit (fast-fail) for 60s. Fallback: return cached response when circuit is open.

Q37. Trade-offs EF Core vs raw ADO.NET

EF Core: rapid development, migrations, change tracking, LINQ — slower for complex bulk operations.
ADO.NET: maximum performance, full SQL control — more code, no migrations, harder to maintain.
Decision: use EF Core default, ADO.NET or Dapper for performance-critical bulk operations.

Q38. Async messaging between microservices — RabbitMQ or Kafka

RabbitMQ: message queuing (work queues, pub/sub, routing). Guarantees delivery, dead-letter queue. .NET via RabbitMQ.Client or MassTransit.

Kafka: event streaming, replay, consumer groups, high throughput. Use when you need event log, replay capability, or 100k+ messages/sec.

Q39. Strategies for inter-service communication in microservices

Sync: HTTP/REST (simple, tight coupling), gRPC (performance, typed contract).

Async: Azure Service Bus (guaranteed), RabbitMQ, Kafka.

Choreography (event-driven, decoupled) vs Orchestration (central coordinator, clearer flow). Choose based on coupling requirements.

Q40. Query tuning in SQL Server — when needed?

Needed when: query runs > 1 second, high CPU/IO in DMVs, blocking/deadlocks, user complaints.

Approach: view Execution Plan → identify scans → add/fix indexes → update statistics → rewrite query (avoid functions on indexed columns) → check missing index recommendations.

Q41. Partitioning large tables

Horizontal partitioning: split rows by date/range. Old partitions archived/compressed. New partitions active.

Query pruning: WHERE clause matches partition key → only relevant partition scanned. Maintenance: switch out old partitions without locking entire table.

Q42. CORS issues — how to handle?

CORS is enforced by browsers, not Postman. Fix: `app.UseCors()` in ASP.NET Core with explicit allowed origins, methods, headers. In APIM: add cors policy. Never use "*" in production — list specific origins. Preflight OPTIONS request must return 200 with CORS headers.

Q43. API consumed by multiple apps — APIM?

APIM provides: different subscription keys per consumer, per-consumer rate limits, transform requests/responses per consumer, analytics per subscription, A/B testing with backends. Without APIM: must manage per-consumer logic in each API.

Q44. Validate requests without writing code — APIM?

APIM policies: `validate-jwt` validates JWT without backend touching it. `validate-parameters` checks query/header parameters. `ip-filter` blocks IPs. `rate-limit` controls traffic. All in XML policy at gateway — backend never touched.

Q45. Difference Sync and Async — give scenario

Sync: `booking.ProcessPayment()` waits until payment API responds (2-3 seconds) before continuing. Thread is blocked during wait.

Async: `await booking.ProcessPaymentAsync()` releases the thread while waiting. Thread handles other requests. 10x more throughput with same hardware.

Q46. Can we use async without await?

Yes: async without await compiles with warning. Method runs synchronously (no actual async). Use case: base class with virtual async methods where derived class may or may not have async work. Also: `Task.FromResult()` to return completed task without await.

Q47. IEnumerable vs IQueryable difference and when to use

`IEnumerable`: executes in-memory. Bring all data then filter.

`IQueryable`: executes on server. Translates LINQ to SQL. Filter on DB server.

Rule: if data source is DB, use `IQueryable` (from `DbSet`). If already in memory, use `IEnumerable`.

Q48. Concurrent list vs normal list

`List<T>`: not thread-safe. Concurrent modifications from multiple threads cause corruption/exceptions.

`ConcurrentQueue<T>`: thread-safe FIFO queue.

`ConcurrentBag<T>`: thread-safe unordered collection.

`ConcurrentDictionary<K,V>`: thread-safe key-value.

For parallel scenarios always use concurrent collections or lock.

Q49. .Result vs ConfigureAwait

`task.Result`: blocks calling thread. Deadlocks in ASP.NET (thread pool thread waiting for task that needs the same thread). Avoid.

`ConfigureAwait(false)`: after awaiting, continue on any thread (not necessarily original context). Use in libraries to avoid deadlocks. In ASP.NET Core (no sync context), less critical but still good practice.

Q50. For Unit Testing — NUnit, xUnit, MSTest

`xUnit`: recommended by Microsoft for .NET Core, no `[TestClass]` needed, `[Fact]` for tests, `[Theory]` for parameterised, isolated test instances.

`NUnit`: mature, `[TestFixture]`, `[Test]`, `[TestCase]`.

`MSTest`: built into VS, `[TestMethod]`.

All support Moq for mocking. AAA pattern: Arrange → Act → Assert.

SECTION 15: AI & ML (13 Questions)

Q1. What is AI Training at a high level?

Training: feeding large amounts of labelled data to a model algorithm. The model adjusts internal weights to minimise prediction error (via backpropagation and gradient descent). Repeated over many epochs until error is acceptably low.

Result: a model file (weights) that can make predictions on new, unseen data.

Q2. Difference between training vs inference.

TRAINING: computationally expensive, runs once (or periodically). Requires GPU clusters. Uses training data to learn patterns. Updates model weights. Takes hours to days.

INFERENCE: lightweight, runs continuously in production. Uses the trained weights to make predictions on new input. Takes milliseconds. Can run on CPU.

Q3. What is an LLM and How Does It Work?

Large Language Model: neural network trained on massive text datasets (internet, books, code). Architecture: Transformer (attention mechanism). Given a sequence of tokens, predicts the most likely next token. Repeated → generates text. Pre-trained on general data, fine-tuned for specific tasks. Examples: GPT-4, Claude, Llama 3.

Q4. Generative AI vs. LLMs

LLM: a TYPE of generative AI specifically for text — generates coherent text, code, analysis.

Generative AI: broader category — includes image generators (DALL-E, Stable Diffusion), video generators, audio generators, code generators. LLM is text-only generative AI. Generative AI can be multi-modal.

Q5. AI Development in .NET

Microsoft.Extensions.AI: abstraction layer for AI services (like IHttpClientFactory for HTTP).

SemanticKernel: orchestration framework for LLMs — manage prompts, plugins, memory, planning, RAG pipelines.

Azure.AI.OpenAI: Azure OpenAI SDK for .NET — call GPT-4, embeddings, DALL-E.

ML.NET: train and serve custom ML models (classification, regression, recommendation) entirely in .NET.

Q6. What is ML.NET?

Microsoft's open-source ML library for .NET. Train and use ML models in C# without Python. Supports: binary/multi-class classification, regression, clustering, anomaly detection, recommendation, image classification. Auto-ML: automatically finds best algorithm and hyperparameters. Model Builder: VS extension with GUI wizard.

Q7. Leveraging Azure AI Services in .NET

Azure OpenAI: GPT-4 completions, embeddings, DALL-E via Azure.AI.OpenAI SDK.

Azure AI Search: vector search for RAG pipelines.

Document Intelligence: extract structured data from PDFs/forms.

Cognitive Services: Vision (image analysis), Speech (STT/TTS), Language (sentiment, NER, translation).

All accessed via Azure SDKs, secured via Managed Identity.

Q8. How to Train an AI Model

1. Collect and label training data.

2. Choose algorithm (linear regression, neural network, transformer).
3. Split data: training (80%), validation (10%), test (10%).
4. Train: feed batches → calculate loss → backpropagate → update weights.
5. Evaluate on validation set — tune hyperparameters.
6. Test on test set — final unbiased evaluation.
7. Deploy trained model for inference.

Q9. Explain AI integration in your project

Structure your answer: Use case → Which Azure AI service → Integration pattern → Data flow → Security.

Example: "We integrated Azure OpenAI for a flight information chatbot. User question → embed via text-embedding-3-small → cosine similarity search in Azure AI Search (our flight documents indexed) → top 3 relevant docs → GPT-4 prompt with context → answer displayed in UI."

Q10. How to call an AI API in .NET

Azure OpenAI SDK:

```
var client = new OpenAIClient(new Uri(endpoint), new AzureKeyCredential(apiKey));
var options = new ChatCompletionsOptions("gpt-4", new[] { new ChatMessage(ChatRole.User, "What is the
weather?") });
var response = await client.GetChatCompletionsAsync(options);
string answer = response.Value.Choices[0].Message.Content;
```

Q11. What is Prompt Engineering

The practice of designing effective inputs (prompts) to get desired outputs from LLMs.

Techniques:

- System prompt: set role and behaviour ("You are a helpful airline booking assistant").
- Few-shot examples: show input/output examples in the prompt.
- Chain-of-thought: ask model to "think step by step".
- Temperature: 0=deterministic, 1=creative.
- Constrain output: "Respond ONLY in JSON format".

Q12. Design AI-powered chatbot in .NET

Architecture: Frontend (Angular) → API (.NET) → Semantic Kernel → Azure AI Search (RAG retrieval) → Azure OpenAI (generation).

SemanticKernel handles: prompt management, memory/context window, plugin orchestration, streaming responses.

Conversation history maintained in session or DB for multi-turn context.

Q13. Chatbot giving wrong answers — how to fix

1. RAG quality: check if retrieved documents are relevant — improve chunking, embedding model, or search relevance.
2. Prompt refinement: add more specific system prompt, add examples.
3. Grounding: instruct model to answer only from provided context ("If answer not in context, say I don't know").
4. Evaluation: run test questions, measure accuracy, use LLM-as-judge.
5. Fine-tuning: if pattern errors persist, fine-tune on domain-specific Q&A dataset.

Azure OpenAI + RAG in .NET

📌 Complete RAG pipeline in .NET with Semantic Kernel:

```
// — Setup —
var builder = Kernel.CreateBuilder();
```



```

// Add Azure OpenAI - chat and embedding models
builder.AddAzureOpenAIChatCompletion(
    deploymentName: "gpt-4",
    endpoint: "https://myopenai.openai.azure.com/",
    credential: new DefaultAzureCredential()); // managed identity

builder.AddAzureOpenAITextEmbeddingGeneration(
    deploymentName: "text-embedding-3-small",
    endpoint: "https://myopenai.openai.azure.com/",
    credential: new DefaultAzureCredential());

var kernel = builder.Build();

// — RAG: Retrieve relevant docs from Azure AI Search —————
public async Task<string> AskWithRAG(string userQuestion, Kernel kernel)
{
    // Step 1: embed the user's question into a vector
    var embedder = kernel.GetRequiredService<ITextEmbeddingGenerationService>();
    var questionVector = await embedder.GenerateEmbeddingAsync(userQuestion);

    // Step 2: search Azure AI Search for semantically similar documents
    var searchClient = new SearchClient(
        new Uri("https://mysearch.search.windows.net"),
        "flight-docs-index",
        new DefaultAzureCredential());

    var searchOptions = new SearchOptions
    {
        VectorSearch = new VectorSearchOptions {
            Queries = { new VectorizedQuery(questionVector.ToArray()) { Fields = {
"contentVector" } } }
        },
        Size = 3 // top 3 most relevant chunks
    };

    var results = await searchClient.SearchAsync<SearchDocument>("", searchOptions);
    var context = new StringBuilder();
    await foreach (var r in results.Value.GetResultsAsync())
        context.AppendLine(r.Document["content"].ToString());

    // Step 3: build prompt with retrieved context
    string systemPrompt = """
        You are a helpful Qatar Airways assistant.
        Answer ONLY from the provided context.
        If the answer is not in the context, say "I don't have that information."
        """;

    string userPrompt = $""""
        Context:
        {context}

        Question: {userQuestion}
        """;

    // Step 4: call GPT-4 with context-augmented prompt
    var chat = kernel.GetRequiredService<IChatCompletionService>();
    var history = new ChatHistory(systemPrompt);
    history.AddUserMessage(userPrompt);

    var response = await chat.GetChatMessageContentAsync(history);
    return response.Content; // grounded, accurate answer
}

```

🔗 *RAG (Retrieval-Augmented Generation) is the key pattern to know for AI integration. It prevents hallucination by grounding the LLM with real retrieved documents. The LLM generates text but is constrained to the provided context.*